

패턴인식(Pattern Recognition)

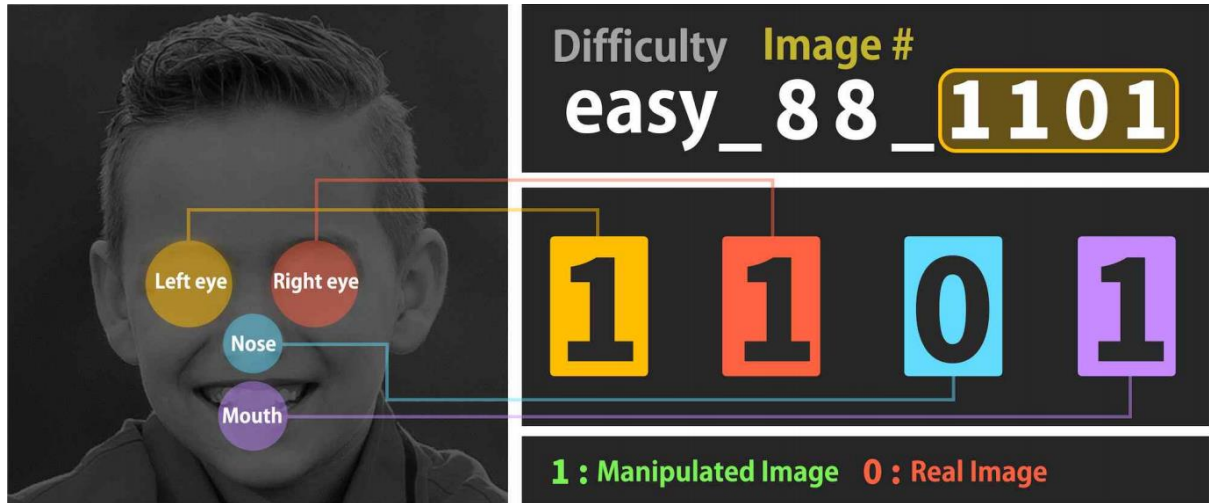
Homework4

제출일	2021.06.21	전공	정보보호학
과목	패턴인식	학번	2020572006
담당교수/조교	문종섭/전승호	이름	이한얼

Contents

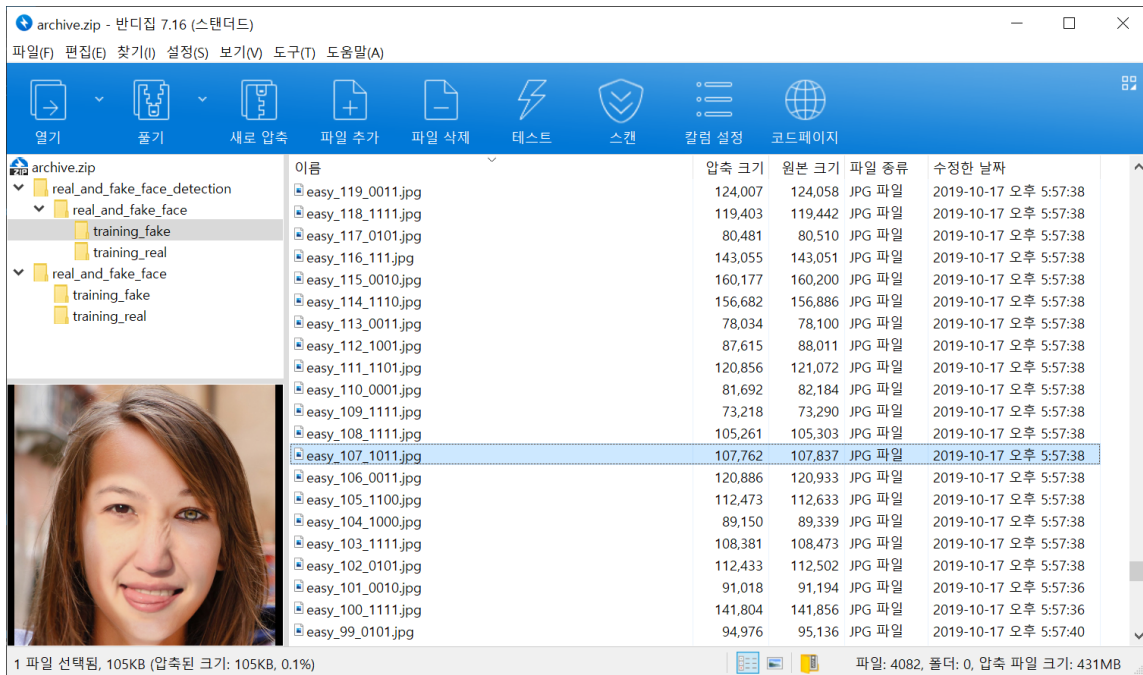
1. Introduction.....	2
1.1 데이터셋 확인	2
2. Related work	4
3. Description of Model	5
3.1 원본/합성 판별 모델	6
3.1.1 레이어 구성	6
3.1.2 모델 학습 결과	6
3.2 합성 위치 예측 모델.....	7
3.2.1 레이어 구성	7
3.2.2 모델 학습 결과	8
4. Experiments.....	8
4.1 원본/합성 판별 모델	8
4.1.1 이미지 어그멘테이션 (Image Augmentation).....	8
4.1.2 모델 레이어 구성 (Model Layer Structure)	9
4.1.3 모델 학습 결과	10
4.1.4 손실 함수 그래프 시각화.....	11
4.2 합성 위치 예측 모델.....	12
4.2.1 데이터 프레임 생성 (Dataframe).....	12
4.2.2 이미지 어그멘테이션 (Image Augmentation).....	13
4.2.3 모델 레이어 구성 (Model Layer Structure)	14
4.2.4 모델 학습 결과	14
4.2.5 손실 함수 그래프 시각화.....	16

1. Introduction



기말 대체 과제로 주어진 3개의 주제 중, 합성 얼굴 이미지 데이터를 이용하여 변조된 이미지와 원본 이미지를 분류하는 모델에 대해 설계, 구현, 튜닝을 진행하였다. 모델은 합성 여부 탐지 모델과 합성 부위 예측 모델로 총 두 개의 모델을 만들었다. 충분히 많지 않은 데이터셋이었기 때문에 어그멘테이션 작업이 필수적으로 필요했고, 성능을 높이기 위한 모델 선택, 레이어 구성 등의 여러 실험과정을 통해 결과적으로 두 모델 모두 약 **97%~99%**의 정확도와 **97%~99%** 사이의 F1-Score를 얻을 수 있었다.

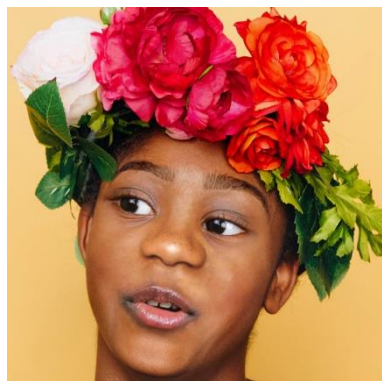
1.1 데이터셋 확인



[그림 1] Real and Fake Face Detection Dataset

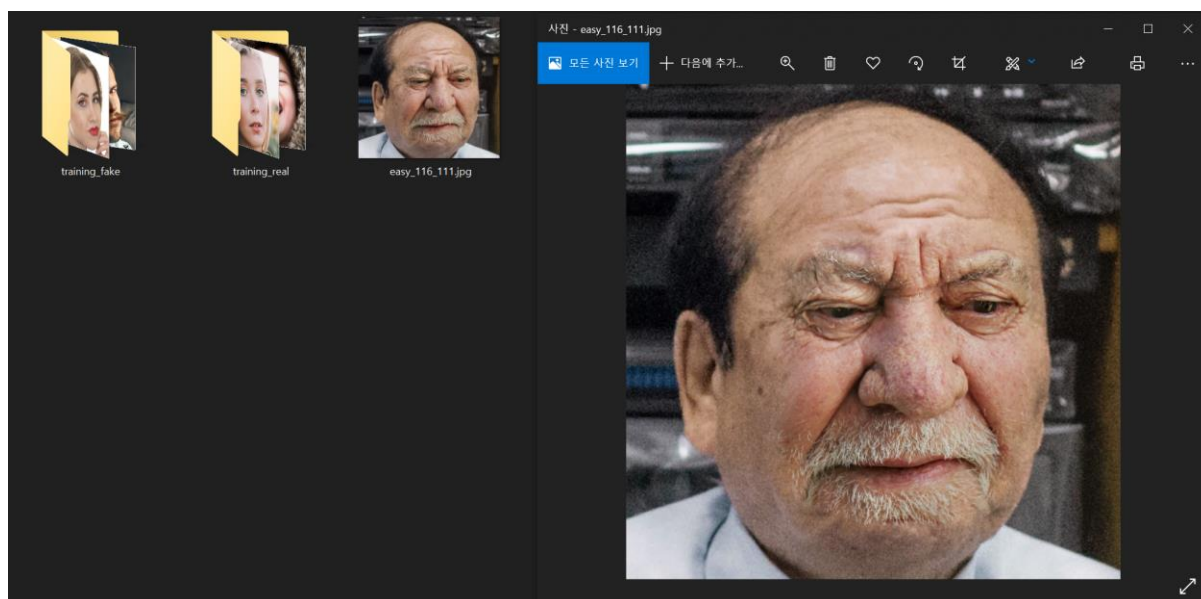
데이터셋은 크게 두 개로 훈련 데이터셋(real_and_fake_face_detection)과 테스트 데이터셋

(real_and_fake_face)으로 구성되어 있고, 이는 각각 다시 합성 이미지인(training_fake)와 원본 이미지인(training_real)로 구성되어 있다. 특히 합성 이미지의 경우 난이도가 easy, mid, hard로 나뉘어져 있으며 mid만 되어도 아래와 같이 시각적으로 합성 여부 탐지가 어려운 것이 특징이다.



[그림 2] mid_389_1101.jpg

라벨링된 이미지를 확인하던 도중 아래와 같이 라벨링이 잘못되어 있어 학습에 사용하기 어려운 이미지를 확인하였고, 따라서 해당 이미지는 학습에서 제외하였다.



[그림 3] 라벨링 오류 데이터셋 (easy_116_111.jpg)

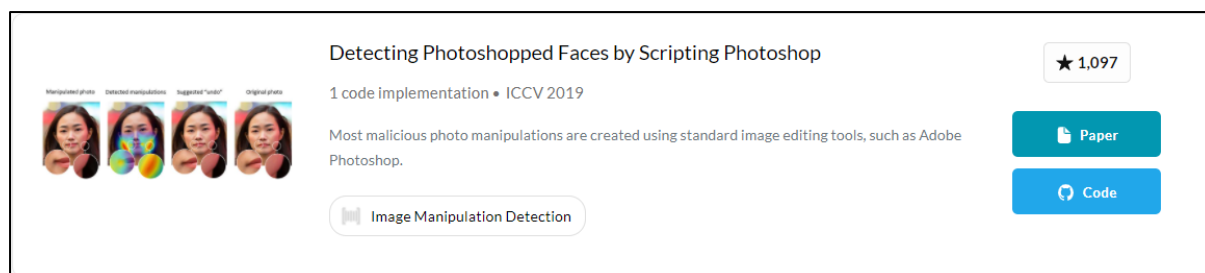
따라서 합성 탐지 모델과 합성 위치 예측 모델을 만들기 위해 아래와 같이 총 4,080장의 데이터셋을 이용하였다.

구분	폴더명	장수	중합	총합
Train	training_fake	959장	2,040장	4,080장
	training_real	1,081장		
Test	training_fake	959장	2,040장	
	training_real	1,081장		

```
128/128 [.....] - 45s 352ms/step - loss: 0.8684 - accuracy: 0.9828 - precision: 0.9834 - recall: 0.9863 - auc: 0.9986 - val_loss: 0.8699 - val_accuracy: 0.9835 - val_precision: 0.9976 - val_recall: 0.9711 - val_auc: 0.9993
Epoch 51/100
128/128 [.....] - 45s 352ms/step - loss: 0.8649 - accuracy: 0.9868 - precision: 0.9844 - recall: 0.9907 - auc: 0.9980 - val_loss: 0.8747 - val_accuracy: 0.9863 - val_precision: 0.9988 - val_recall: 0.9376 - val_auc: 0.9988
Epoch 52/100
128/128 [.....] - 45s 352ms/step - loss: 0.8465 - accuracy: 0.9936 - precision: 0.9944 - recall: 0.9935 - auc: 0.9998 - val_loss: 0.8698 - val_accuracy: 0.9884 - val_precision: 0.9894 - val_recall: 0.9734 - val_auc: 0.9984
Epoch 53/100
128/128 [.....] - 45s 351ms/step - loss: 0.8636 - accuracy: 0.9838 - precision: 0.9843 - recall: 0.9852 - auc: 0.9987 - val_loss: 0.8634 - val_accuracy: 0.9884 - val_precision: 0.9849 - val_recall: 0.9788 - val_auc: 0.9986
Epoch 54/100
128/128 [.....] - 45s 352ms/step - loss: 0.8488 - accuracy: 0.9917 - precision: 0.9917 - recall: 0.9926 - auc: 0.9996 - val_loss: 0.8863 - val_accuracy: 0.9688 - val_precision: 0.9976 - val_recall: 0.9434 - val_auc: 0.9992
Epoch 55/100
128/128 [.....] - 45s 351ms/step - loss: 0.8544 - accuracy: 0.9843 - precision: 0.9870 - recall: 0.9833 - auc: 0.9989 - val_loss: 0.8844 - val_accuracy: 0.9755 - val_precision: 0.9578 - val_recall: 0.9777 - val_auc: 0.9993
```

2. Related work

관련 연구를 찾기 위해 SOTA (State Of the Art) 연구를 모아둔 Paperswithcode라는 사이트에서 fake face detection과 관련된 자료를 찾던 도중 아래와 같은 연구를 확인할 수 있었다.



[그림 4] 합성/변조 이미지 탐지 SOTA 모델

제공받은 데이터 셋이 GAN으로 만들어진 것이 아니기에 위의 포토샵 기반의 탐지 연구가 적합하다 판단하였다.

	Train	Val	Test
Source	OpenImage & Flickr	Flickr	
Total Images	1.1M	10k	100
Unmanipulated images	157k	5k	50
Manipulated images	942k	5k	50
Manipulations	Random FAL	Pro Artist	

[그림 5] 데이터셋 구성

위 연구에서 실험을 위해 OpenImage와 Flickr에서 데이터셋을 마련하였으며 크게 변조되지 않은 이미지와 변조된 이미지로 구성하여 훈련, 검증, 테스트 셋으로 나누었다. 변조 방법의 경우 Random FAL(일종의 자동화 스크립트)과 Pro Artist가 변조하는 방법을 사용하였다. 과제를 위해 제공받은 데이터셋과 위의 연구에서 마련한 데이터셋이 거의 동일하여 학습에 도움이 될 것이라 판단하였다.

저자는 실험을 위해 크게 두 가지 모델을 만들었다. 이는 얼굴이 합성되었는지를 여부를 식별하는 글로벌 분류 모델과 합성된 위치를 예측하는 로컬 분류 모델이다. 따라서 과제를 진행하기 위해 만든 모델 또한 위 연구를 기반으로 합성 탐지 여부 모델과 합성 부위 예측 모델을 만들었다.

저자들은 실험을 위해 ImageNet 데이터셋으로부터 Pretrained된 DRN(Dilated Residual Network)의 변형인 DRN-C-26 모델을 사용하였다. 이를 통해 얻은 실험 결과는 다음과 같다.

Algorithm			Validation (Random FAL)					Test (Professional Artist)				
Method	Resol- ution	with Aug?	Accuracy			AP	2AFC	Accuracy			AP	2AFC
			Total	Orig	Mod			Total	Orig	Mod		
Chance	–	–	50.0	50.0	50.0	50.0	50.0	50.0	50.0	50.0	50.0	50.0
Human	–	–	–	–	–	–	53.5	–	–	–	–	71.1
FaceForensics++ [31]	–	–	51.3	86.3	16.2	52.7	–	50.0	85.7	14.3	55.3	61.9
Self-consistency* [15]	–	–	–	–	–	53.7	–	–	–	–	56.4	72.0
Low-res no aug.	400		97.0	97.2	96.9	99.7	99.5	89.0	86.0	92.0	96.8	98.0
Low-res with aug.	400	✓	93.7	91.6	95.7	98.9	98.9	83.0	74.0	92.0	94.4	96.0
High-res with aug.	700	✓	97.1	99.8	94.5	99.8	100.0	90.0	96.0	84.0	97.4	98.0

[그림 6] Real or fake classifier performance

저자들은 모델에 대해 스크립트를 통해 랜덤으로 변조한 Random FAL 방식과 프로 아티스트가 변조한 Professional Artist 방식에 대해 모델의 성능을 측정하였다. 성능 측정 결과, 고해상도의 인풋이 들어가면 더 높은 성능을 보이는 것을 확인하였다. 또한 augmentation이 없이 학습된 것이 더 뛰어난 것을 보였으나 augmentation을 적용한 것이 반대로 조금 더 모델이 robust한 결과를 보인다 하였다.

3. Description of Model

위 관련 연구의 실험에 기반하여 이미지의 원본/합성 판별을 위한 모델과 합성된 위치를 예측하는 모델로 크게 두 개를 구현하였다. Pretrained Model을 사용하지 않고 학습을 위해 두 모델 모두 이미지 처리에 효율적이라 판단되는 Convolution Neural Network를 사용하였다.

3.1 원본/합성 판별 모델

3.1.1 레이어 구성

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 300, 300, 32)	896
conv2d_1 (Conv2D)	(None, 300, 300, 32)	9248
max_pooling2d (MaxPooling2D)	(None, 150, 150, 32)	0
conv2d_2 (Conv2D)	(None, 150, 150, 64)	18496
conv2d_3 (Conv2D)	(None, 150, 150, 64)	36928
max_pooling2d_1 (MaxPooling2D)	(None, 75, 75, 64)	0
batch_normalization (Batch Normalization)	(None, 75, 75, 64)	256
conv2d_4 (Conv2D)	(None, 75, 75, 128)	73856
max_pooling2d_2 (MaxPooling2D)	(None, 37, 37, 128)	0
batch_normalization_1 (Batch Normalization)	(None, 37, 37, 128)	512
conv2d_5 (Conv2D)	(None, 37, 37, 256)	295168
max_pooling2d_3 (MaxPooling2D)	(None, 18, 18, 256)	0
batch_normalization_2 (Batch Normalization)	(None, 18, 18, 256)	1024
flatten (Flatten)	(None, 82944)	0
dense (Dense)	(None, 128)	10616960
dense_1 (Dense)	(None, 1)	129

=====
Total params: 11,053,473
Trainable params: 11,052,577
Non-trainable params: 896

[그림 7] 원본/합성 판별 모델 (syn_detect_model)

모델에 입력되는 이미지의 사이즈는 원본의 경우 600x600이나 원본 그대로 모델 학습시켜본 결과 시간이 오래 걸린다는 단점이 있어 300x300으로 축소하여 모델의 Input으로 지정하였다. 이후 Convolution 2D, Max Pooling, Batch Normalization, flatten, Dense 레이어로 모델을 구성하였고 출력 부분에는 sigmoid 함수를 통해 이진 분류가 가능하도록 하였다.

3.1.2 모델 학습 결과

```
[*] Start fitting the model
Epoch 1/100
128/128 [=====] - 55s 389ms/step - loss: 0.9013 - accuracy: 0.5162 - precision: 0.5440 - recall: 0.5375 - auc: 0.5191 - val_loss: 0.7792
Epoch 2/100
128/128 [=====] - 46s 358ms/step - loss: 0.7030 - accuracy: 0.6147 - precision: 0.6300 - recall: 0.6614 - auc: 0.6502 - val_loss: 1.0505
Epoch 3/100
128/128 [=====] - 46s 358ms/step - loss: 0.6426 - accuracy: 0.6377 - precision: 0.6457 - recall: 0.7012 - auc: 0.6959 - val_loss: 1.2785
Epoch 4/100
128/128 [=====] - 49s 384ms/step - loss: 0.6188 - accuracy: 0.6642 - precision: 0.6886 - recall: 0.6688 - auc: 0.7296 - val_loss: 0.6334
Epoch 5/100
128/128 [=====] - 46s 363ms/step - loss: 0.5777 - accuracy: 0.6877 - precision: 0.6944 - recall: 0.7336 - auc: 0.7666 - val_loss: 0.6875
```

학습 초반에는 위와 같이 loss 값이 0.9013, accuracy 0.5162로 시작하는 것을 확인할 수 있었고,

학습 종료 시점에는 아래와 같이 loss 값이 0.0479, accuracy가 0.9931임을 확인할 수 있었고 추가적인 다른 metric을 통해서 학습이 전반적으로 잘 이루어져 있음을 확인할 수 있었다.

```
Epoch 53/100
128/128 [=====] - 46s 360ms/step - loss: 0.0594 - accuracy: 0.9858 - precision: 0.9870 - recall: 0.9861 - auc: 0.9989 - val_loss: 0.0829 - val_accuracy: 0.9749
Epoch 54/100
128/128 [=====] - 46s 361ms/step - loss: 0.0597 - accuracy: 0.9863 - precision: 0.9870 - recall: 0.9870 - auc: 0.9985 - val_loss: 0.0598 - val_accuracy: 0.9884
Epoch 55/100
128/128 [=====] - 48s 375ms/step - loss: 0.0451 - accuracy: 0.9926 - precision: 0.9935 - recall: 0.9926 - auc: 0.9997 - val_loss: 0.1890 - val_accuracy: 0.9112
Epoch 56/100
128/128 [=====] - 46s 360ms/step - loss: 0.0559 - accuracy: 0.9873 - precision: 0.9880 - recall: 0.9880 - auc: 0.9990 - val_loss: 0.0548 - val_accuracy: 0.9896
Epoch 57/100
128/128 [=====] - 46s 361ms/step - loss: 0.0531 - accuracy: 0.9868 - precision: 0.9871 - recall: 0.9880 - auc: 0.9990 - val_loss: 0.0579 - val_accuracy: 0.9859
Epoch 58/100
128/128 [=====] - 46s 360ms/step - loss: 0.0479 - accuracy: 0.9931 - precision: 0.9954 - recall: 0.9917 - auc: 0.9996 - val_loss: 0.0559 - val_accuracy: 0.9865
```

3.2 합성 위치 예측 모델

3.2.1 레이어 구성

```
Model: "sequential_1"
-----
Layer (type)                 Output Shape              Param #
-----
conv2d_6 (Conv2D)            (None, 300, 300, 32)     896
-----
conv2d_7 (Conv2D)            (None, 300, 300, 32)     9248
-----
max_pooling2d_4 (MaxPooling2 (None, 150, 150, 32)     0
-----
conv2d_8 (Conv2D)            (None, 150, 150, 64)     18496
-----
conv2d_9 (Conv2D)            (None, 150, 150, 64)     36928
-----
max_pooling2d_5 (MaxPooling2 (None, 75, 75, 64)      0
-----
batch_normalization_3 (Batch (None, 75, 75, 64)      256
-----
dropout (Dropout)            (None, 75, 75, 64)       0
-----
conv2d_10 (Conv2D)           (None, 75, 75, 128)      73856
-----
max_pooling2d_6 (MaxPooling2 (None, 37, 37, 128)     0
-----
batch_normalization_4 (Batch (None, 37, 37, 128)     512
-----
dropout_1 (Dropout)           (None, 37, 37, 128)      0
-----
conv2d_11 (Conv2D)           (None, 37, 37, 256)      295168
-----
max_pooling2d_7 (MaxPooling2 (None, 18, 18, 256)     0
-----
batch_normalization_5 (Batch (None, 18, 18, 256)     1024
-----
flatten_1 (Flatten)           (None, 82944)             0
-----
dense_2 (Dense)               (None, 128)               10616960
-----
dense_3 (Dense)               (None, 15)                 1935
-----
Total params: 11,055,279
Trainable params: 11,054,383
Non-trainable params: 896
```

[그림 8] 합성 위치 예측 모델 (loc_detect_model)

합성 위치 예측 모델 또한 원본/합성 탐지 모델과 거의 동일한 구조를 가지고 있다. 차이점은 학습 과정에서 빠른 과적합이 이루어지는 것을 확인하였다. 따라서 Dropout 레이어를 추가하였고 출력 부분에는 라벨을 15개로 지정하여 softmax 함수를 통해 출력되도록 하였다. 라벨에 대한 구체적인 내용은 실험 부분에 기술하였다.

3.2.2 모델 학습 결과

학습 종료 시점에 아래와 같이 loss 값이 0.0586, accuracy가 0.9875임을 확인할 수 있었고 추가적인 다른 metric을 통해서 학습이 전반적으로 잘 이루어져 있음을 확인할 수 있었다.

```
Epoch 56/100
68/68 [=====] - 27s 450ms/step - loss: 0.0534 - accuracy: 0.9927 - precision: 0.9969 - recall: 0.9906 - auc: 1.0000 - val_loss: 0.1446 - val_accuracy: 0.9648
Epoch 57/100
68/68 [=====] - 26s 428ms/step - loss: 0.0642 - accuracy: 0.9917 - precision: 0.9927 - recall: 0.9885 - auc: 0.9993 - val_loss: 0.2147 - val_accuracy: 0.9492
Epoch 58/100
68/68 [=====] - 27s 451ms/step - loss: 0.0487 - accuracy: 0.9958 - precision: 0.9969 - recall: 0.9948 - auc: 0.9999 - val_loss: 0.2530 - val_accuracy: 0.9297
Epoch 59/100
68/68 [=====] - 26s 429ms/step - loss: 0.0597 - accuracy: 0.9875 - precision: 0.9947 - recall: 0.9844 - auc: 1.0000 - val_loss: 0.1338 - val_accuracy: 0.9727
Epoch 60/100
68/68 [=====] - 28s 461ms/step - loss: 0.0493 - accuracy: 0.9937 - precision: 0.9958 - recall: 0.9906 - auc: 1.0000 - val_loss: 0.1537 - val_accuracy: 0.9635
Epoch 61/100
68/68 [=====] - 30s 502ms/step - loss: 0.0586 - accuracy: 0.9875 - precision: 0.9895 - recall: 0.9844 - auc: 0.9999 - val_loss: 0.1669 - val_accuracy: 0.9688
```

4. Experiments

4.1 원본/합성 판별 모델

4.1.1 이미지 어그멘테이션 (Image Augmentation)

어그멘테이션을 진행하지 않고 원본/합성 탐지 모델을 학습한 결과 충분한 성능이 도출되지 않았다. 이에 따라 어그멘테이션 작업이 필요로 했고, 아래와 같은 옵션을 사용하여 어그멘테이션을 진행하였다.

구분	옵션	값	비고
훈련 데이터셋	horizontal_flip	True	수평 플립
훈련 데이터셋	vertical_flip	False	수직 플립
훈련 데이터셋	shear_range	0.2	기울기 비율
훈련 데이터셋	zoom_range	0.2	확대 비율
훈련 데이터셋	rescale	1 / 255.	이미지 스케일링
테스트 데이터셋	rescale	1 / 255.	이미지 스케일링
테스트 데이터셋	validation_split	0.2	훈련/평가 분할 비율

모델의 Robustness를 위해 수평 플립을 True로 설정하였고 기울기와 확대 비율에 대해 주로 사용되는 값인 0.2를 설정하였다. 또한 훈련 데이터셋과 테스트 데이터셋에 대해 각각 이미지 스케일

링을 적용하였고 테스트 데이터셋에 대해서는 0.2 비율로 평가와 테스트 데이터셋으로 나누었다. 어그멘테이션 옵션을 적용한 결과는 다음과 같다.

```
Found 2040 images belonging to 2 classes.  
Found 1633 images belonging to 2 classes.  
Found 407 images belonging to 2 classes.
```

Training 데이터셋 2,040장 Validation 데이터셋 1,633장 Test 데이터셋 407장에 대해 어그멘테이션을 진행하였다.

4.1.2 모델 레이어 구성 (Model Layer Structure)

```
52 def _build_layer(self):  
53     layers = [  
54         Conv2D(32, (3, 3), padding='same', input_shape=(300, 300, 3), activation='relu'),  
55         Conv2D(32, (3, 3), padding='same', activation='relu'),  
56         MaxPool2D(pool_size=(2, 2)),  
57  
58         Conv2D(64, (3, 3), padding='same', activation='relu'),  
59         Conv2D(64, (3, 3), padding='same', activation='relu'),  
60         MaxPool2D(pool_size=(2, 2)),  
61         BatchNormalization(),  
62  
63         Conv2D(128, (3, 3), padding='same', activation='relu'),  
64         MaxPool2D(pool_size=(2, 2)),  
65         BatchNormalization(),  
66  
67         Conv2D(256, (3, 3), padding='same', activation='relu'),  
68         MaxPool2D(pool_size=(2, 2)),  
69         BatchNormalization(),  
70  
71         Flatten(),  
72         Dense(units=128, activation='relu'),  
73         Dense(units=1, activation='sigmoid')]  
74  
75     return layers
```

[그림 9] 레이어 구성

6개의 Convolution 2D 레이어와 3개의 Maxpooling 2D 레이어, 그리고 3개의 Batch Normalization 레이어를 구성하고 출력부분에는 활성화 함수로 sigmoid를 사용하여 합성 여부를 탐지할 수 있도록 만들었다. Dropout의 경우 모델이 과적합되지 않아, 별도로 적용하지 않았다. 모델의 옵티마이저는 RMSProp을 사용하였고 learning rate 값으로 0.00001을 사용하였다.

모델 구성의 순서에서 CS231n 강의 기준으로 Conv2D → BatchNorm → MaxPool2D 순으로 쌓는 것이 적합하다 하였으나 Conv2D → MaxPool2D → BatchNorm 형태로 구성하는 것이 더 학습 성

능이 좋은 것을 실험적으로 확인하였다.

4.1.3 모델 학습 결과

100회의 에폭을 통해 학습한 결과 아래와 같다.

```
Epoch 53/100
128/128 [=====] - 46s 360ms/step - loss: 0.0594 - accuracy: 0.9858 - precision: 0.9870 - recall: 0.9861 - auc: 0.9989 - val_loss: 0.0829 - val_accuracy: 0.9749
Epoch 54/100
128/128 [=====] - 46s 361ms/step - loss: 0.0597 - accuracy: 0.9863 - precision: 0.9870 - recall: 0.9870 - auc: 0.9985 - val_loss: 0.0598 - val_accuracy: 0.9884
Epoch 55/100
128/128 [=====] - 48s 375ms/step - loss: 0.0451 - accuracy: 0.9926 - precision: 0.9935 - recall: 0.9926 - auc: 0.9997 - val_loss: 0.1890 - val_accuracy: 0.9112
Epoch 56/100
128/128 [=====] - 46s 360ms/step - loss: 0.0559 - accuracy: 0.9873 - precision: 0.9880 - recall: 0.9880 - auc: 0.9990 - val_loss: 0.0548 - val_accuracy: 0.9896
Epoch 57/100
128/128 [=====] - 46s 361ms/step - loss: 0.0531 - accuracy: 0.9868 - precision: 0.9871 - recall: 0.9880 - auc: 0.9990 - val_loss: 0.0579 - val_accuracy: 0.9859
Epoch 58/100
128/128 [=====] - 46s 360ms/step - loss: 0.0479 - accuracy: 0.9931 - precision: 0.9954 - recall: 0.9917 - auc: 0.9996 - val_loss: 0.0559 - val_accuracy: 0.9865
```

[그림 10] 학습 과정

Early Stop을 적용하여 58번째 에폭에서 종료되었고, 점진적으로 loss가 줄고, accuracy가 증가하는 양상을 확인할 수 있었다. 부가적으로 precision, recall, auc에 대한 평가 metric을 추가하여 확인할 수 있도록 하였다. 학습된 모델에 대해 validation set으로 평가한 결과 아래와 같다.

```
[*] Start evaluating the model
103/103 [=====] - 10s 101ms/step - loss: 0.0559 - accuracy: 0.9865 - precision: 0.9941 - recall: 0.9803 - auc: 0.9984

[+] Loss : 0.0559
[+] Accuracy : 0.9865
[+] Precision : 0.9941
[+] Recall : 0.9803
[+] AUC : 0.9984
```

[그림 11] 모델 Evaluation 결과

결과 값으로 Loss값 5.6%, Accuracy값 98.7%, Precision 99.4%, Recall 98.0%, AUC 99.9%을 확인할 수 있다.

```
[*] Real : easy_108_1111.jpg
[*] Fake : easy_109_1111.jpg
[*] Fake : easy_10_0001.jpg
[*] Fake : easy_110_0001.jpg
[*] Fake : easy_111_1101.jpg
[*] Fake : easy_112_1001.jpg
[*] Fake : easy_113_0011.jpg
[*] Fake : easy_114_1110.jpg
[*] Fake : easy_115_0010.jpg
[*] Fake : easy_117_0101.jpg
[*] Fake : easy_118_1111.jpg
[*] Fake : easy_119_0011.jpg
[*] Fake : easy_11_1111.jpg
[*] Fake : easy_120_0011.jpg
[*] Real : easy_121_0011.jpg
[*] Fake : easy_122_1011.jpg
[*] Real : easy_123_0100.jpg
```

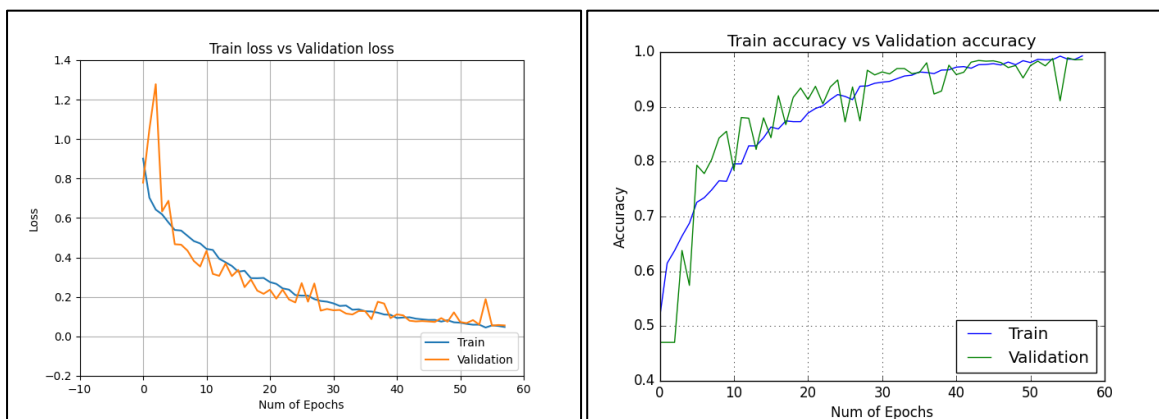
[그림 12] Inference 과정

최종적으로 합성 여부 탐지를 위해 Test 데이터셋의 모든 fake 이미지를 넣은 결과 위와 같이 대부분의 이미지가 올바르게 Inference되는 것을 확인할 수 있고, 일부 Real로 오분류 되는 것 또한 확인할 수 있다. Inference를 통해 최종적인 F1 Score의 경우 98.8%의 성능을 보임을 확인하였다.

```
[*] F1 Score: 0.9874869655891554
```

[그림 13] F1 Score

4.1.4 손실 함수 그래프 시각화



모델 학습의 초반부에 변화가 심한 양상을 보이기도 하나 점차 변화 폭이 좁아지면서 안정적으로 학습되는 것을 확인할 수 있다. Loss는 약 0.9에서 시작하여 0.05까지 줄어들며, Accuracy는 약 0.55부터 시작하여 0.99에 근접하게 학습되는 것을 확인할 수 있다.

4.2 합성 위치 예측 모델

4.2.1 데이터 프레임 생성 (Dataframe)

합성 여부와 달리 합성 위치에 대한 예측 모델을 만들기 위해서는 별도의 formal한 구조로 만들어 두어야 학습에 용이할 것이라 판단되어 아래와 같이 판다스(Pandas) 라이브러리를 이용하여 데이터프레임을 생성하였다.

	filename	left_eye	right_eye	nose	mouth	label
0	easy_100_1111.jpg	1	1	1	1	1111
1	easy_101_0010.jpg	0	0	1	0	0010
2	easy_102_0101.jpg	0	1	0	1	0101
3	easy_103_1111.jpg	1	1	1	1	1111
4	easy_104_1000.jpg	1	0	0	0	1000
...	...	...	...	...	...	...
954	mid_96_1111.jpg	1	1	1	1	1111
955	mid_97_1111.jpg	1	1	1	1	1111
956	mid_98_1111.jpg	1	1	1	1	1111
957	mid_99_1100.jpg	1	1	0	0	1100
958	mid_9_1110.jpg	1	1	1	0	1110

959 rows × 6 columns

[그림 14] Dataframe 생성

파일 이름의 합성 부위를 나타내는 4자리의 숫자만 추출하여 각각의 필드로 나타내었다. 특이한 점은 label이라는 컬럼을 만들었는데 이는 기존 left_eye, right_eye, nose, mouth와 같이 별개의 컬럼을 라벨로 지정하여 학습을 시도한 결과, 방법론을 잘못 적용한 것인지 학습 결과가 좋지 않았다. 마지막 레이어에 sigmoid 또는 softmax로 Unit을 4로 설정하여도 학습 결과가 미미한 것을 확인하였다. 전반적으로 모델 설계 방식에 대한 방법론 조사를 시도했으나 크게 찾지 못하였다. 하지만 조사 도중 일부 예시에서 위와 같이 개개 별로 나누는 것이 아닌 하나로 통합하여 라벨로 두는 방법을 찾아 있는 것을 확인하였고, 이를 적용하였더니 학습 결과가 개선되는 것을 확인할 수 있었다.

4.2.2 이미지 어그멘테이션 (Image Augmentation)

합성 여부 탐지 모델과 달리 변조 부위 탐지 모델의 경우 합성된 이미지만 사용해야 했기에 충분한 학습을 위해 이미지 어그멘테이션 작업을 진행하였다. 다음은 이미지 어그멘테이션을 위해 사용한 옵션의 종류와 값을 나타내었다.

구분	옵션	값	비고
학습 데이터셋	horizontal_flip	False	수평 플립
학습 데이터셋	vertical_flip	False	수직 플립
학습 데이터셋	shear_range	0.2	기울기 비율
학습 데이터셋	zoom_range	0.2	확대 비율
테스트 데이터셋	validation_split	0.2	훈련/평가 분할 비율

왼쪽 눈과 오른쪽 눈의 예측 위치가 바뀌어선 되지 않기에 수평 플립을 False로 주었고, 적은 데이터셋이기에 학습 데이터셋의 일부를 수직 플립하는 옵션 또한 False로 적용하였다. 나머지 옵션인 기울기, 확대, 분할 비율의 경우 일반적으로 0.2를 설정한다고 판단하여 값을 설정하였다.

```
Found 959 validated image filenames belonging to 15 classes.  
Found 768 validated image filenames belonging to 15 classes.  
Found 191 validated image filenames belonging to 15 classes.
```

Training Set의 경우 총 959개를 사용하였고, Validation Set과 Test Set은 0.2 비율로 각각 768개, 191개로 나누어 어그멘테이션을 진행하였다.

4.2.3 모델 레이어 구성 (Model Layer Structure)

```
222 def _build_layer(self):
223
224     layers = [
225         Conv2D(32, (3, 3), padding='same', input_shape=(300, 300, 3), activation='relu'),
226         Conv2D(32, (3, 3), padding='same', activation='relu'),
227         MaxPool2D(pool_size=(2, 2)),
228
229         Conv2D(64, (3, 3), padding='same', activation='relu'),
230         Conv2D(64, (3, 3), padding='same', activation='relu'),
231         MaxPool2D(pool_size=(2, 2)),
232         BatchNormalization(),
233         Dropout(0.2),
234
235         Conv2D(128, (3, 3), padding='same', activation='relu'),
236         MaxPool2D(pool_size=(2, 2)),
237         BatchNormalization(),
238         Dropout(0.2),
239
240         Conv2D(256, (3, 3), padding='same', activation='relu'),
241         MaxPool2D(pool_size=(2, 2)),
242         BatchNormalization(),
243
244         Flatten(),
245         Dense(units=128, activation='relu'),
246         Dense(units=15, activation='softmax')]
247
248     return layers
```

합성 여부 탐지 모델과 동일하게 레이어를 구성하였으며 달라진 부분은 출력 부분에 합성의 경우의 수인 15가지에 대해 softmax 함수를 적용한 것이다. 15가지는 합성 부위 별로 나올 수 있는 경우의 수인 2^4 에서 합성되지 않은 0000에 대한 경우를 제외한 것이다. 또한 Dropout 레이어를 추가하여 학습이 과적합 되지 않도록 하였다. Dropout을 0.1로 모두 적용하여 진행한 결과 적용하지 않은 것과 비슷하게 대부분의 평가 메트릭에서 99%대를 확인할 수 있었고, Dropout을 0.2로 적용한 결과 조금 더 느슨하게 학습되는 것을 확인할 수 있었다. 다음은 모델학습 결과이다.

4.2.4 모델 학습 결과

```
Epoch 56/100
60/60 [=====] - 27s 450ms/step - loss: 0.0534 - accuracy: 0.9927 - precision: 0.9969 - recall: 0.9906 - auc: 1.0000 - val_loss: 0.1446 - val_accuracy: 0.9648
Epoch 57/100
60/60 [=====] - 26s 428ms/step - loss: 0.0642 - accuracy: 0.9917 - precision: 0.9927 - recall: 0.9885 - auc: 0.9993 - val_loss: 0.2147 - val_accuracy: 0.9492
Epoch 58/100
60/60 [=====] - 27s 451ms/step - loss: 0.0487 - accuracy: 0.9958 - precision: 0.9969 - recall: 0.9948 - auc: 0.9999 - val_loss: 0.2530 - val_accuracy: 0.9297
Epoch 59/100
60/60 [=====] - 26s 429ms/step - loss: 0.0597 - accuracy: 0.9875 - precision: 0.9947 - recall: 0.9844 - auc: 1.0000 - val_loss: 0.1338 - val_accuracy: 0.9727
Epoch 60/100
60/60 [=====] - 28s 461ms/step - loss: 0.0493 - accuracy: 0.9937 - precision: 0.9958 - recall: 0.9906 - auc: 1.0000 - val_loss: 0.1537 - val_accuracy: 0.9635
Epoch 61/100
60/60 [=====] - 30s 502ms/step - loss: 0.0586 - accuracy: 0.9875 - precision: 0.9895 - recall: 0.9844 - auc: 0.9999 - val_loss: 0.1669 - val_accuracy: 0.9688
```

[그림 15] 모델 학습 결과 (epoch: 61 종료)

61번째 에폭에서 Early stop이 적용되어 종료되었다. 점진적으로 loss가 줄고, accuracy가 증가하는 양상을 확인할 수 있었고, 부가적으로 precision, recall, auc에 대한 평가 metric을 추가하여 확인할

수 있도록 하였다. (그림의 가시성을 위해 val_precision, val_recall, val_auc 정보를 담지 않았다)

모델에 대해 Validation Set을 사용하여 평가한 결과 다음과 같다.

```
[*] Start evaluating the model
48/48 [=====] - 7s 138ms/step - loss: 0.1669 - accuracy: 0.9688 - precision: 0.9850 - recall: 0.9388 - auc: 0.9996

[+] Loss : 0.1669
[+] Accuracy : 0.9688
[+] Precision : 0.985
[+] Recall : 0.9388
[+] AUC : 0.9996
```

[그림 16] 모델 Evaluation 결과

모델 성능 평가 결과 값으로 Loss 값 16%, Accuracy 값 96.9%, Precision 99.0%, Recall 94.0%, AUC 99.9%를 확인할 수 있다. 다음으로 학습한 합성 부위 예측 탐지 모델에 대해 합성 이미지 전체를 입력으로 넣어 추론을 진행하는 과정을 아래와 같이 진행하였다.

```
[*] mid_46_1111.jpg : 14, 14
[+] 원 : True,
[+] 오 : True
[+] 코 : True
[+] 입 : True
[*] mid_470_1001.jpg : 8, 8
[+] 원 : True,
[+] 오 : False
[+] 코 : False
[+] 입 : True
[!] mid_471_1111.jpg : 14, 13 --> Fail
[*] mid_472_1101.jpg : 12, 12
[+] 원 : True,
[+] 오 : True
[+] 코 : False
[+] 입 : True
[*] mid_473_0011.jpg : 2, 2
[+] 원 : False,
[+] 오 : False
[+] 코 : True
[+] 입 : True
```

[그림 17] Inference 진행 과정

0 ~ 15사이의 값을 0001, 0010, 0011 ~ 1111 값으로 딕셔너리로 아래와 같이 매핑 해두었다.

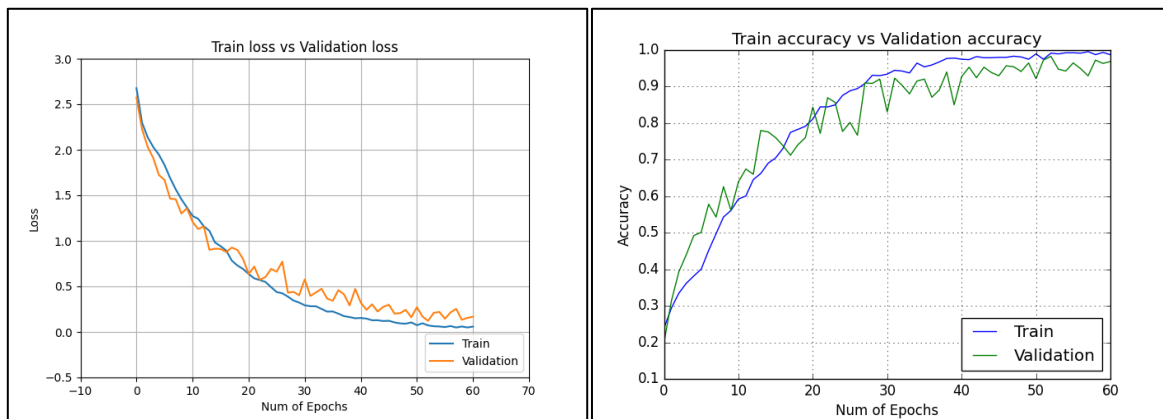
```
{'00001': 0, '0010': 1, '0011': 2, '0100': 3, '0101': 4, '0110': 5, '0111': 6, '1000': 7, '1001': 8, '1010': 9, '1011': 10, '1100': 11, '1101': 12, '1110': 13, '1111': 14}
```

실제 합성 이미지가 들어 갔을 때 실제 값과 예측 값을 출력하였으며 만약 다를 경우 Fail이라는 문자열을 출력하도록 하여 구분하도록 하였다. 올바르게 합성 부위를 탐지한 결과의 경우 부위를 합성 부위를 True/False로 표현하여 확인할 수 있도록 하였다. 전체 959장의 합성 이미지에서 Inference를 통해 F1 Score를 계산한 결과는 다음과 같다.

[*] F1 Score: 0.9708

Inference 진행 결과, F1 Score 97.0%로 학습이 매우 잘 이루어졌음을 확인할 수 있다.

4.2.5 손실 함수 그래프 시각화



합성 여부 탐지 모델과 달리 초반 변화폭이 큰 양상이 없으며 전반적으로 loss 값이 줄어 들고 accuracy가 증가하는 추세를 확인할 수 있다. 이와 같은 추세와 F1 Score를 통해 모델 학습 잘 이루어졌음을 알 수 있다.